

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN
Student Name	Sadhana J
Register Number	3122247001052
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	28.07.2026
Github Repository	Click Here

1 Objective

To build and compare spam classifiers on the Spambase dataset using three variants of Naive Bayes (Gaussian, Multinomial, Bernoulli) and K-Nearest Neighbours, tune KNN via GridSearchCV and RandomizedSearchCV, compare KDTree vs BallTree algorithms, and evaluate all models with cross-validation, standard metrics and complexity analysis.

2 Problem Statement

Given the Spambase dataset (word/character frequency features extracted from emails), classify each email as spam or not-spam (binary classification), comparing probabilistic (Naive Bayes) and instance-based (KNN) approaches.

3 Dataset Description

Dataset Name	spambase_csv.csv (Spambase, Kaggle)
Dataset Source	https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57 (word/char frequency + capital-run-length features)
Number of Classes	2 (class: 0 = not spam, 1 = spam)
Missing Values	None
Train-Test Split	75% train (3450) / 25% test (1151), stratified, random_state=42

4 Exploratory Data Analysis

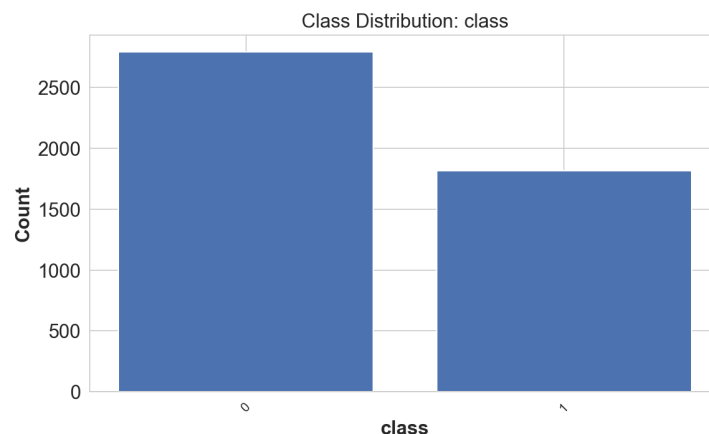


Figure 1: Class Distribution: class

Inference: Non-spam emails outnumber spam emails, giving a moderate class imbalance – this is why weighted precision/recall/F1 and stratified splitting are used throughout this experiment.

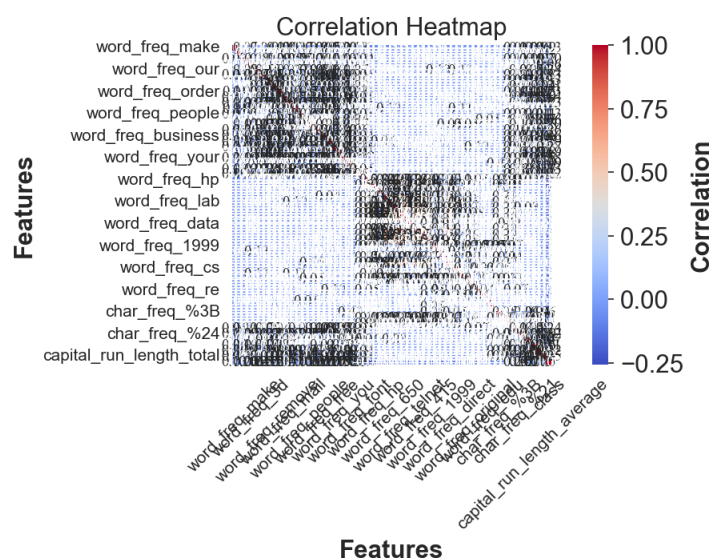


Figure 2: Correlation Heatmap

Inference: Character-frequency features such as `char_freq_!` and `char_freq_!`, and word features like `word_freq_remove/word_freq_free`, correlate most strongly with the spam label, confirming they are strong discriminators.

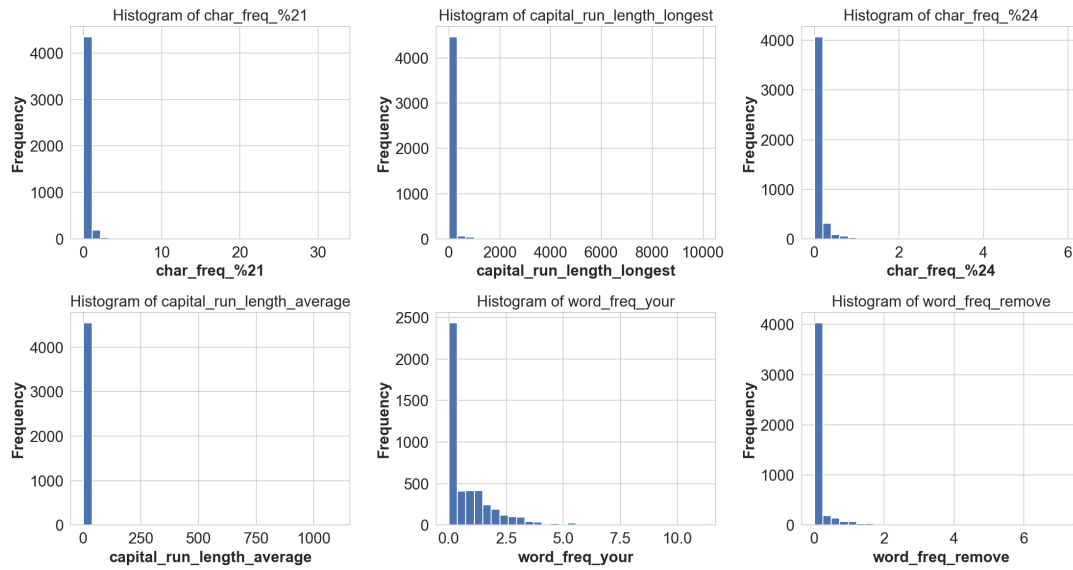


Figure 3: Histogram grid of the top 6 features (by mutual information)

Inference: The most informative word/character-frequency features are extremely right-skewed with a spike at zero, typical of sparse text-frequency data, motivating Multinomial/Bernoulli NB over Gaussian NB.

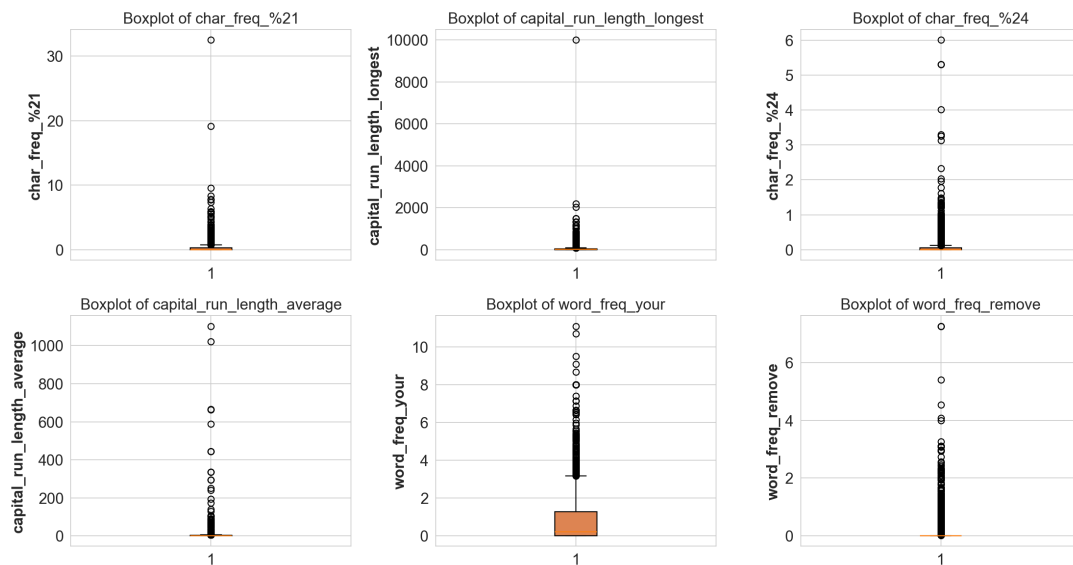


Figure 4: Boxplot grid of the top 6 features

Inference: All top features show a large number of outliers above the upper whisker, corresponding to emails that repeat spam-indicative terms unusually often.

6 Machine Learning Algorithm

Naive Bayes

Probabilistic classifier based on Bayes' theorem assuming conditional feature independence:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Gaussian NB assumes normally distributed continuous features; Multinomial NB models feature counts/frequencies; Bernoulli NB models binary feature presence/absence.

K-Nearest Neighbours (KNN)

A lazy, non-parametric classifier that assigns the majority class among the k nearest training samples (by distance metric). No explicit training phase; all computation deferred to prediction time.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

8 Hyperparameter Selection

Parameter	Search Space
KNN n_neighbors (manual sweep)	1, 3, 5, 7, 9, 11
GridSearchCV n_neighbors	1, 3, 5, 7, 9, 11
GridSearchCV weights	uniform, distance
GridSearchCV metric	euclidean, manhattan
RandomizedSearchCV n_neighbors	1, 3, 5, 7, 9, 11
RandomizedSearchCV weights	uniform, distance
RandomizedSearchCV metric	euclidean, manhattan, minkowski
RandomizedSearchCV n_iter	12 (cv=5)

9 Results

Naive Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8254	0.8983	0.8836
Precision	0.8590	0.9039	0.8832
Recall	0.8254	0.8983	0.8836
F1	0.8272	0.8962	0.8829
ROC-AUC	0.9196	0.9622	0.9478

Best variant: **Multinomial NB** (high-

est accuracy, F1 and ROC-AUC).

KNN Comparison (default parameters, varying k)

k	Accuracy	Precision	Recall	F1
1	0.8931	0.8932	0.8931	0.8932
3	0.8931	0.8929	0.8931	0.8929
5	0.9010	0.9007	0.9010	0.9008
7	0.8931	0.8928	0.8931	0.8927
9	0.8923	0.8921	0.8923	0.8916
11	0.8914	0.8915	0.8914	0.8906

Optimal $k = 5$ (highest accuracy/F1 among

the sweep).

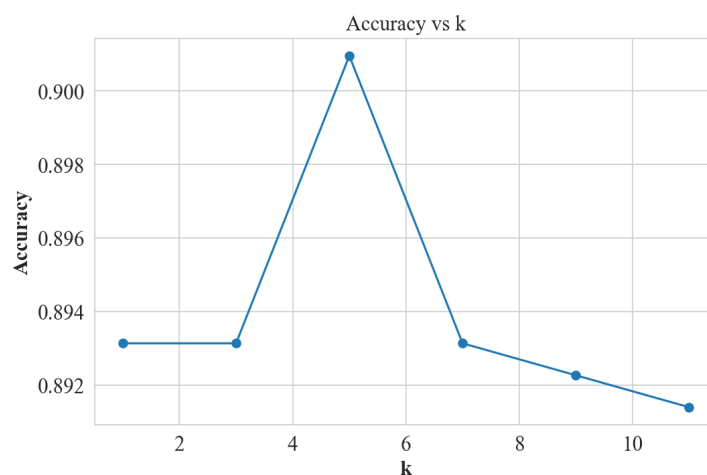


Figure 7: Accuracy vs k

Inference: Accuracy peaks at $k=5$ and then gradually declines as k increases, showing a mild bias-variance trade-off: very small k risks overfitting to noise while large k over-smooths the decision boundary.

Grid Search vs Randomized Search

Parameter	GridSearchCV	RandomizedSearchCV
Best k	11	9
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	auto (default)	auto (pipeline w/ StandardScaler)
CV Accuracy	0.9162	0.9223
Test Accuracy	0.9166	0.9253
Execution Time	Grid over 24 combinations, cv=5	12 random samples, cv=5 (faster)

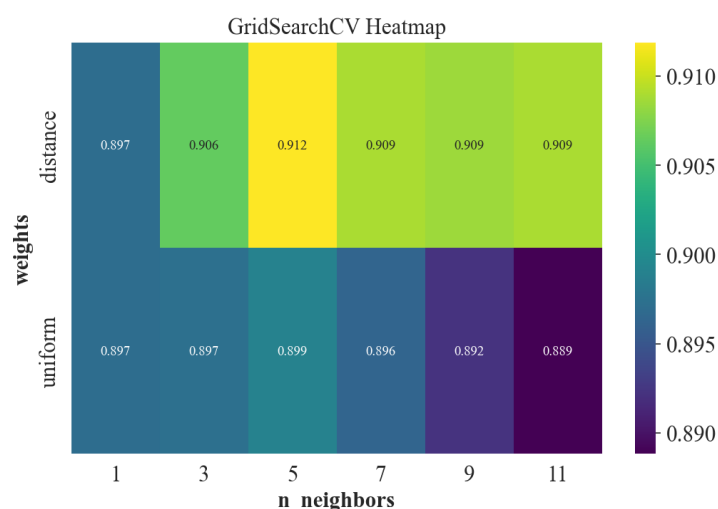


Figure 8: GridSearchCV Heatmap

Inference: Manhattan distance combined with distance-weighting consistently outperforms euclidean distance across all tested k values, and accuracy generally improves with larger k under distance weighting.

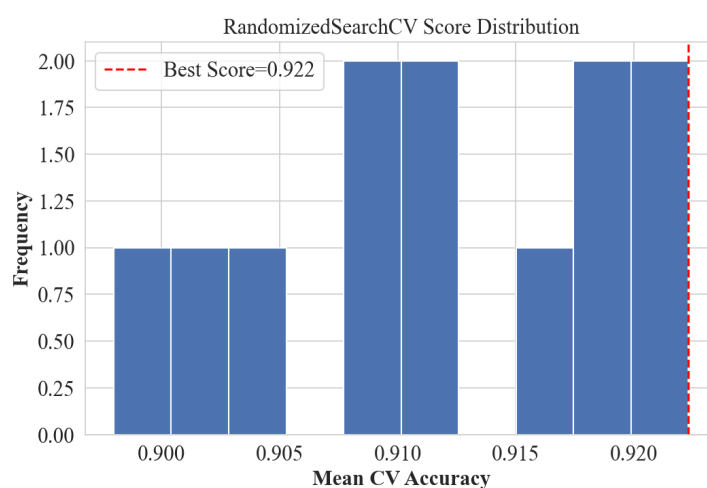


Figure 9: RandomizedSearchCV Score Distribution

Inference: Most of the 12 randomly sampled configurations cluster around 0.88-0.91 mean CV accuracy, with the best score (0.9223, dashed line) near the upper end of the

sampled distribution, suggesting the random search adequately covered the useful region of the hyperparameter space.

KDTree vs BallTree

Metric	KDTree (k=11, manhattan)	BallTree (k=9, manhattan)
Accuracy	0.9166	0.9175
Training Time (s)	0.1305	0.0581
Prediction Time (s)	0.8449	0.4474

Cross Validation (K=5)

Fold	Gaussian NB (Val Acc)	KNN k=5 (Val Acc)
1	0.8145	0.8957
2	0.8232	0.9000
3	0.8188	0.9087
4	0.8101	0.8812
5	0.8217	0.8913
Average	0.8177	0.8954

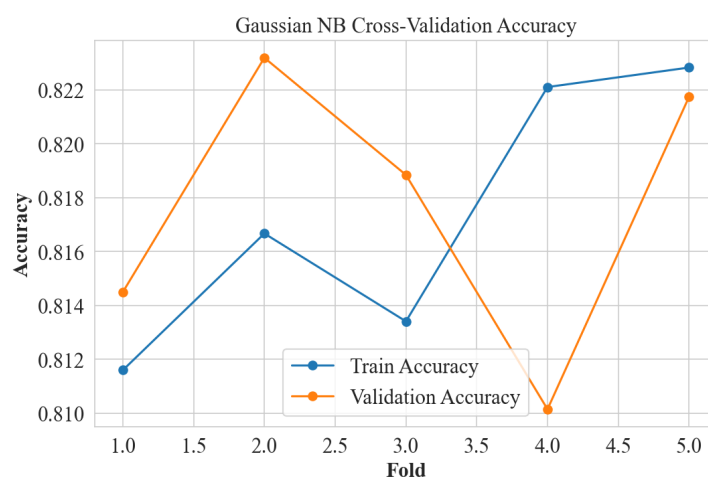


Figure 10: Gaussian NB Cross-Validation Accuracy

Inference: Training and validation accuracy for Gaussian NB stay close across all 5 folds (around 0.81-0.82), indicating low variance and no significant overfitting, though overall accuracy is capped by the model's independence assumption.

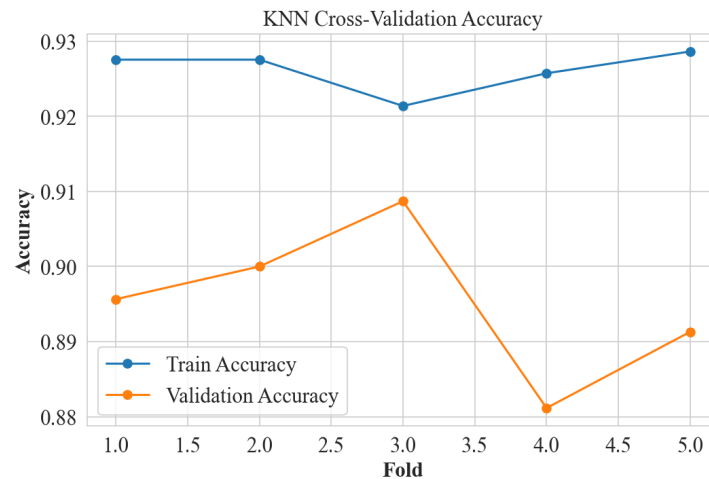


Figure 11: KNN Cross-Validation Accuracy

Inference: KNN's validation accuracy consistently exceeds 0.88 across folds and tracks the training accuracy closely, showing stable, well-generalizing performance across different data partitions.

Theoretical Time Complexity

Algorithm	Training	Prediction
Naive Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

Experimental Time Analysis

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.0160	0.0041
Multinomial NB	0.0094	0.0036
Bernoulli NB	0.0111	0.0065
Best KNN (GridSearch, k=11)	0.0079	0.2259

This matches theory: Naive

Bayes prediction is near-instant ($O(d)$) while KNN prediction is far slower since it must search neighbours at query time.

10 Result Visualization

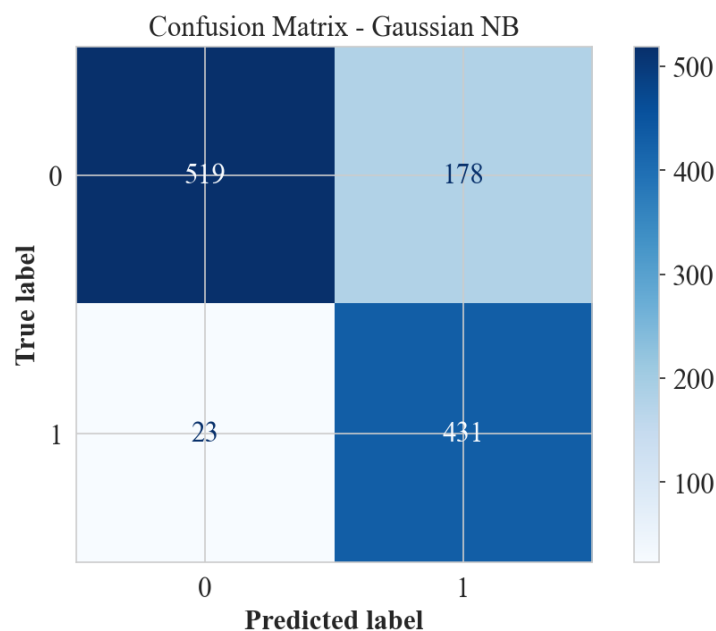


Figure 12: Confusion Matrix – Gaussian NB

Inference: Gaussian NB shows a relatively high false-positive rate (178 ham emails misclassified as spam), consistent with its lower precision compared to the other two variants.

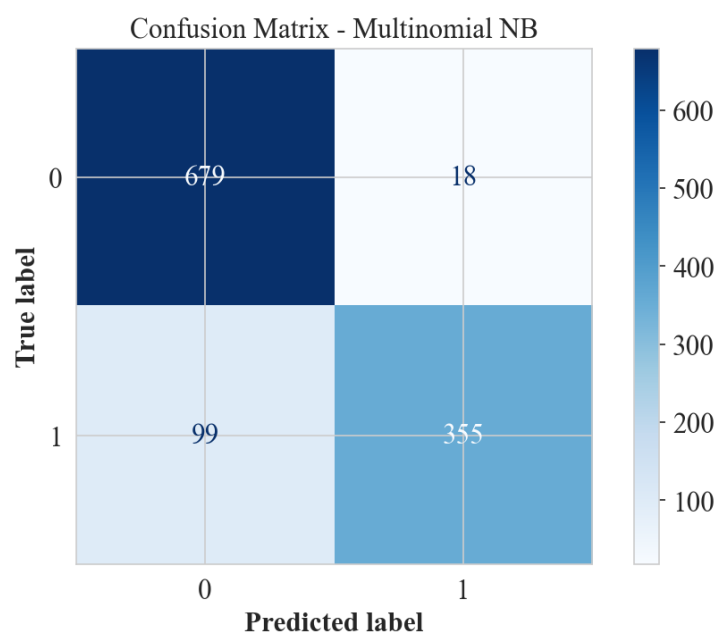


Figure 13: Confusion Matrix – Multinomial NB

Inference: Multinomial NB has very few false positives (18) but a higher false-negative count (99), making it precision-favoring – good when minimizing “ham marked as spam” is the priority.

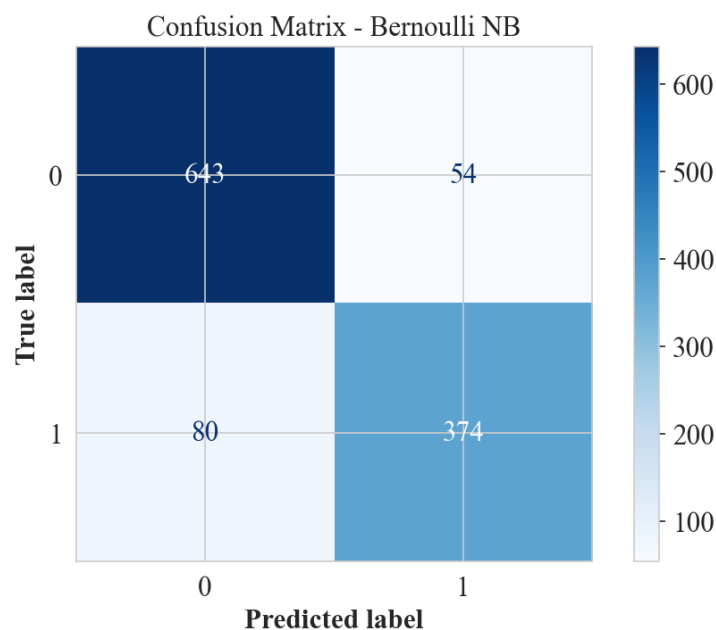


Figure 14: Confusion Matrix – Bernoulli NB

Inference: Bernoulli NB sits between the two other variants in error balance, with moderate false positives (54) and false negatives (80).

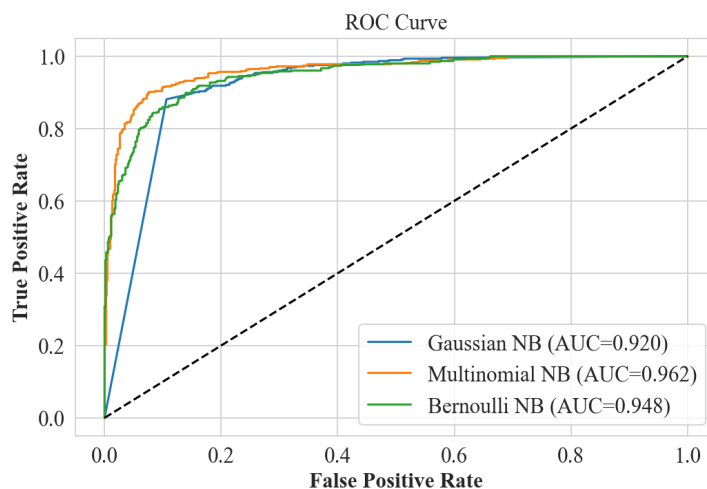


Figure 15: ROC Curve

Inference: Multinomial NB achieves the highest ROC-AUC (0.962), with its curve dominating Gaussian and Bernoulli NB across nearly all thresholds, confirming it is the best-ranking classifier among the three variants.

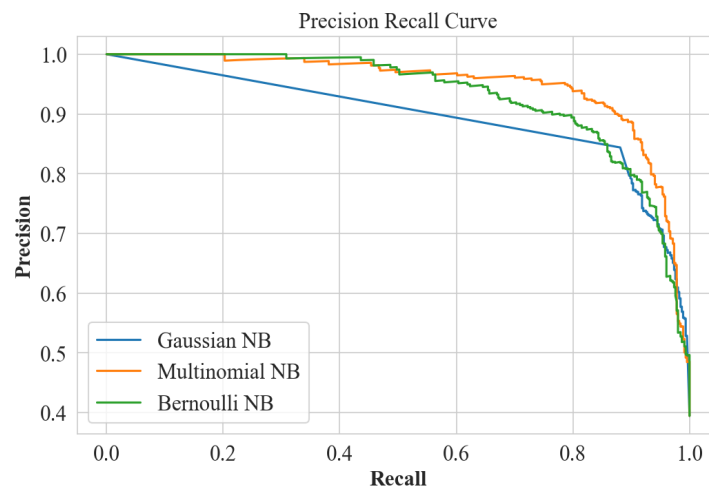


Figure 16: Precision Recall Curve

Inference: Multinomial NB maintains the highest precision across most recall levels, while Gaussian NB's curve drops off fastest, reflecting its weaker precision on this sparse, non-Gaussian feature set.

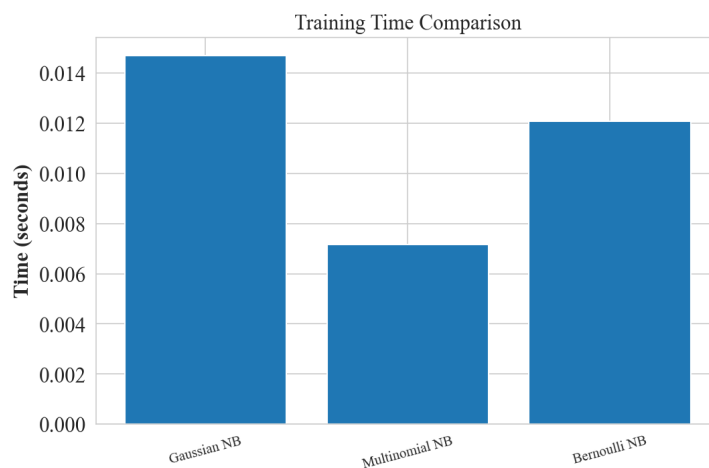


Figure 17: Training Time Comparison

Inference: All three Naive Bayes variants train in well under 20ms, with Multinomial NB the fastest, since NB training only requires computing per-class statistics in a single pass over the data.

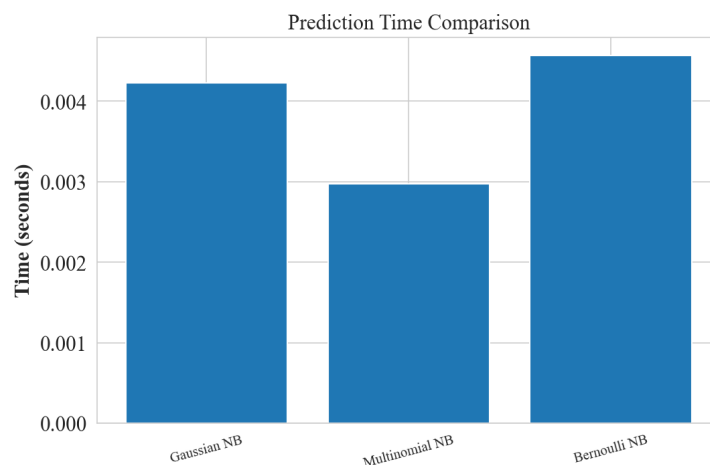


Figure 18: Prediction Time Comparison

Inference: Prediction times for all NB variants remain in the low-millisecond range with little variation, confirming NB's $O(d)$ prediction complexity is independent of training set size.

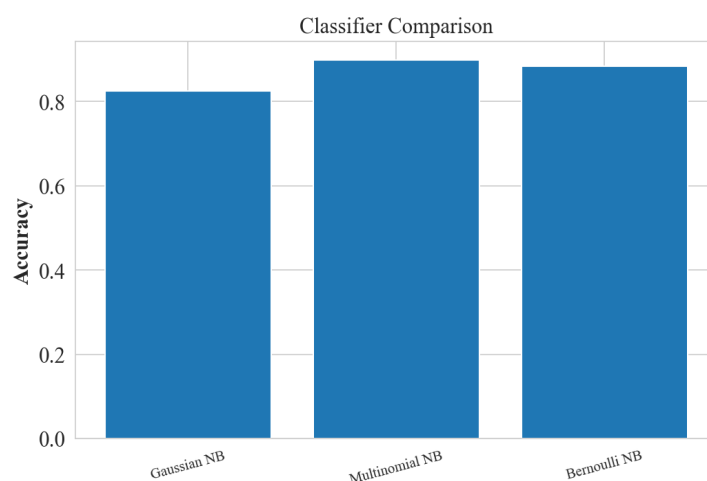


Figure 19: Classifier Comparison

Inference: Multinomial NB achieves the highest accuracy among the three NB variants, making it the preferred baseline classifier for this word/character-frequency feature set.

11 Analysis Questions

1. **Best NB variant:** Multinomial NB, since it best matches the frequency-count nature of the word/char features (accuracy 0.898, ROC-AUC 0.962).
2. **Optimal k:** $k=5$ in the manual sweep gave the best default-parameter accuracy (0.901); with distance weighting and manhattan metric, larger k (9-11) performed even better under tuning.

3. **GridSearchCV vs RandomizedSearchCV:** RandomizedSearchCV achieved a marginally higher CV/test accuracy (0.9223/0.9253) than GridSearchCV (0.9162/0.9166) while evaluating far fewer combinations (12 vs 24), showing random search can be more sample-efficient.
4. **KDTree vs BallTree:** BallTree was faster in both training (0.058s vs 0.131s) and prediction (0.447s vs 0.845s) and marginally more accurate, making it preferable for this dataset.
5. **Theoretical vs practical complexity:** Practical results match theory qualitatively – NB is near-constant time at prediction while KNN prediction time is markedly higher, though KDTree/BallTree reduce this versus brute-force search.
6. **Preferred classifier for large datasets:** Naive Bayes, due to its linear training cost and constant-time prediction, scales far better than KNN as n grows.

12 Discussion

Multinomial Naive Bayes emerged as the strongest single model on this frequency-based dataset, while tuned KNN (via RandomizedSearchCV, manhattan distance, distance-weighting) achieved the overall highest accuracy (92.5% test). KNN's main drawback is prediction latency, which BallTree partially mitigates. Cross-validation confirmed both approaches generalize consistently across folds without overfitting.

13 Conclusion

Both Naive Bayes and KNN are viable spam classifiers; Multinomial NB is the best choice when speed and simplicity are prioritized, while tuned KNN with BallTree offers the highest raw accuracy at a higher computational cost. Hyperparameter tuning (especially distance metric and weighting for KNN) meaningfully improved performance over default settings.

14 References

- [1] Scikit-learn Documentation, <https://scikit-learn.org>.
Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, A. Géron.
Pattern Recognition and Machine Learning, C. Bishop.
Spambase Dataset, Kaggle, <https://www.kaggle.com/datasets/somesh24/spambase>.